Observing the Edge: Host Behavior as a Foundation for Workflow Security

Sofia Vaz[1,2][009−0000−2976−3608], Vitor A. Cunha[1,2][0000−0002−4566−0198], and Rui L. Aguiar[1,2][000−0003−0107−6253]

[1] Departamento de Electrónica, Telecomunicações e Informática, Universidade de Aveiro, Campus Universitário de Santiago, 3810-193 Aveiro, Portugal
[2] Instituto de Telecomunicações, Universidade de Aveiro, Campus Universitário de Santiago, 3810-193 Aveiro, Portugal

Abstract. Network Management and Operations are becoming increasingly more complex and rely on different types of workflows. To cope with the increased complexity, workflows are becoming more distributed, with tasks running closer to the elements they affect, leveraging edge environments for locality and other hardware for performance (such as GPUs for analytics or MLOps). Therefore, securing the infrastructure that hosts them becomes a foundational concern. In these dynamic and often heterogeneous systems, traditional methods of ensuring host trustworthiness can be impractical or incomplete. This paper proposes a lightweight behavior-based security solution designed to protect workflow execution by continuously observing the host environment. Instead of relying on static trust anchors or external attestation, our system monitors runtime behavior at the host level and triggers alerts when predefined rules are violated. These rules alert to clear security issues, such as unauthorized process activity, unusual file accesses, or network behavior irregularities. By anchoring workflow security in host observation, our approach enables early detection of compromised or misbehaving edge nodes. We evaluate the system in simulated edge deployments and demonstrate its ability to detect a range of security-relevant events with low overhead, offering a practical foundation for securing workflows through host-side observation.

Keywords: Observability · Workflows · Edge Computing · Cybersecurity .

1 Introduction

The centralized computing model dominated computer networking until the late twentieth century, when a shift toward more distributed architectures began [1]. This shift introduced the idea of deploying computing resources closer to end-users, rather than relying solely on distant centralized data centers [2]. This concept laid the foundation for edge computing, a paradigm that brings computing and data storage closer to end-users, reducing the dependency on centralized cloud servers [3]. Edge computing improves data processing speed and efficiency,

offering significant benefits in terms of lower latency, reduced bandwidth usage, and lower power consumption [4, 5].

At the same time, networks have ballooned in complexity and most operations now are neither manual nor executed on a single machine [6]. Instead, many networks employ their operations via Network Function Virtualization (NFV). However, NFV requires orchestration, which is in itself a workflow [7, 8].

However, when executing network-level operations, it is vital to ensure that they are executed correctly, as any deviation from the expected results can have disastrous consequences [9]. Moreover, to ensure that the workflow is safe, especially in edge environments, it is vital to ensure that the host that runs it is also safe [10].

However, the lack of a centralized control system and reduced visibility into distributed nodes make edge networks vulnerable to various threats. Compromised or malicious nodes can exploit these vulnerabilities, leading to unauthorized access, data breaches, service disruptions, or the compromise of the network workflow. This paper addresses these security concerns by exploring the implementation of an observation system in hosts. This is, of course, with the express goal of securing the environment in which workflows run, which in term secures the workflow itself.

The objective of this paper is to propose an observation system for orchestrated systems, with the main goal of giving administrators better tools to manage their distributed workflows. The following section, section 2, focuses on a study of the current state of the art. Then, it is possible to design (section 3) and implement the security system (section 4). The results are then obtained and discussed (section 5). Finally, conclusions are drawn in section 6.

2 State of the Art

Edge observability, a developing area of research, is hindered by the scarcity of real-world implementations of edge computing [11, 12, 13]. This scarcity has led to a lack of tailored security solutions for edge environments, a situation exacerbated by the telecommunications industry's prioritization of economic viability over security considerations in deploying edge computing infrastructure. As a result, many efforts are being made to repurpose cloud-based solutions for edge scenarios, further highlighting the urgent need for edge-specific security solutions.

However, this approach is flawed. For example, [14] adapts cloud monitoring tools for edge applications, but it is clear that edge-specific monitoring is still in its infancy. This adaptation fails to address edge-specific issues, as cloud solutions inherently lack the capabilities to manage the complexities of edge environments, prompting us to question the current approach. Others [15, 16] have approached the problem in a similar way, but came to the same conclusions.

Recent efforts have emerged to address the limitations of cloud-based solutions. For example, [17] proposes a monitoring framework that uses FMonE

and Kafka instances to monitor metrics such as CPU, memory, disk, and network usage at the host and container levels. Although this approach demonstrates robustness and minimal overhead, it is crucial to note that it focuses on metric-based monitoring, neglecting the urgent need for direct security-specific observability.

Other approaches, such as [18], organize nodes into peer-to-peer networks and employ FogMon to obtain metrics. However, the paper lacks detailed information on the specific metrics and their security implications, limiting the understanding of its effectiveness in addressing security concerns.

More advanced approaches, such as [19], introduce self-adaptive monitoring based on dynamic rules that adjust metric collection rates according to system state, improving resource efficiency and monitoring accuracy. Nevertheless, these methods revolve around resource metrics rather than directly monitoring security events. In contrast, [20] employs a decentralized "gossiping" approach, where the nodes share status information to maintain a synchronized view across the network. Although this approach is reliable, it mainly covers resource metrics, leaving potential security threats inferred rather than explicitly monitored.

Other works, such as [21], leverage SNMP protocols with REST APIs to monitor distributed systems, focusing on metrics visibility rather than security. Similarly, [22] monitors various layers in distributed environments, but uses different tools across layers, potentially missing interlayer security interactions. In edge-cloud continuum scenarios, [23] monitors the quality of service at the application level, suitable for both the cloud and edge but limited in providing comprehensive system security visibility.

The predominant reliance on resource metrics poses a significant limitation for security in edge environments. Security events are often inferred from indirect indicators, such as abnormal CPU usage, which can lead to misinterpretation of the system state. For example, real-world security threats, such as the xz backdoor [24], have shown that subtle system impacts, such as slightly slower SSH connections, can be overlooked without direct behavioral monitoring. Therefore, there is a clear need for edge-specific observability solutions that focus on direct monitoring of security-related behaviors to ensure robust protection against threats in edge computing environments.

3 Architecture

Edge computing is promising for applications that require low latency. Its distinctive feature, which gives it its name, is that computing takes place at the edge of the network, meaning that it is no longer centralized.

Although there are many edge computing paradigms, the current work focuses on Multi-access Edge Computing (MEC). This paradigm settles on the idea that there is a centralized MEC Orchestrator (MEO), which orchestrates a cluster located closer to clients. This cluster contains the MEC hosts, which will run the MEC applications. MEC applications are developed by software providers that may or may not have a long-term relationship with infrastruc-

ture owners and other MEC stakeholders. Thus, there is no assurance that the application will work as expected. Not only may there be malicious intent from the developers, genuine mistakes also exist. Thus, it is not a question of whether there are security issues; it is a question of when, how, and who finds them. There is a very real possibility that an attacker could leverage these security issues to compromise the host, which in term compromises whatever workflows are meant to run there, including the orchestration workflow itself.

The MEO is the bridge between the core of the system and its hosts, and, per MEC design, there is little to no observability into hosts. Thus, there is a chance that malicious code is running on a host, and there is little to no visibility into what it is precisely doing. In addition, the MEC hosts communicate directly, which means that there is no mediation in terms of unsafe communication. That gives any malicious actor a direct communication link with any host, as long as they have access to one. They may, for example, use that link to exploit host or application vulnerabilities, which in term could grant them control over the victim host as well. Hosts being compromised, however, is extremely dangerous, especially when taking into account MEC's critical use cases. This vector of attack may sound unlikely, as a bad actor would need to have an active host in the cluster. However, there is no physical barrier, nor are there many virtual barriers to become a MEC host. A host may join the cluster without necessarily being verified as safe.

3.1 System Architecture

The monitoring solution's architecture should be flexible, as all MEC hosts have differing needs, and therefore monitoring must be approached differently. In order to maximize the usability of the solution, this must be taken into account. Moreover, systems evolve over time, meaning that host observability and requirements may change as well. Thus, a modular architecture is a clear step forward.

There is a clear need for two specific modules: the event detector and the event aggregator. The event detector is necessary to observe the system. It should be able to detect and alert to actions that take place both natively and in containerized settings. Moreover, the events for which it alerts should be customizable.

The second necessary module, the event aggregator, is necessary because events should be visible at the MEO level. If not, then events will be scattered on the various hosts being monitored. In order to function as an aggregator, however, it should subscribe to all instances of detectors, or poll them. It is important to note that the aggregator should not run on any monitored node, as it may use resources necessary for other processes. Instead, it should run on the MEO, or even on an entirely different machine. This aggregator could exist in the form of a traditional API, or even as a timeseries. The crucial point is to ensure that the aggregator captures and stores data throughout the monitoring system's operation.

Considering that the scenario in which the system will be deployed refers to MEC, there are other considerations that must be taken into account. All components must be controlled and deployed by the MEO. As its name implies, its role is to orchestrate the system. Considering that all other applications will be controlled by the MEO, it makes sense for it to also control the observability system.

The existence of these modules, and the fact that the MEO should control all of them, has some consequences in terms of communication. The orchestrator must have communication with the cluster controller, as that is the bridge that all other MEC applications are expected to use. Then, the orchestrator will be able to "give the order" for the cluster controller to deploy the monitoring system in all nodes in the cluster.

The theorized architecture is present in Figure 1.

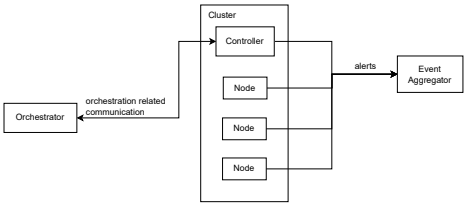

Fig. 1. Solution reference architecture

4 Implementation

As discussed in section 3, there are two main components: the orchestrator and the cluster in which the application will run. An event detector will run on every cluster node, and an event aggregator will also exist.

The first exploratory implementation includes that of an orchestrated solution with a single-node cluster. It will work as a test for the system's utility in an edge environment and a baseline for comparison. With utility ensured, a multi-node solution can be explored. It is an orchestrated solution with two nodes on its cluster — a Virtual Machine (VM) and a bare metal machine. To add further noise to this scenario, the bare metal machine is in a different network. It has operating system and hardware specifications that differ from the other VMs. The goal is to test the extensibility of the solution regarding the number of nodes and their nature. Most real deployments will have larger clusters with varied node specifications, so the multi-node solution aims to approximate to those settings.

The tool used as the event detector is Falco [3]. This tool allows us to create observation rules, meaning that it is possible to cater to whatever the system requires.

The orchestrator plays a crucial role in any MEC approximation, managing the deployment and operation of the system. For the current work, the one chosen was Open Source MANO (OSM). However, other approaches may use different orchestrators.

4.1 Scenario 1: Single Node Implementation

Three things are necessary to deploy an instance in OSM: the cluster, an Network Slice (NS) package, and a Virtualized Network Function (VNF) package. In this case, the cluster is single-node, meaning it only comprises of a singular machine. It is a VM, and after it was set up as a cluster controller, its necessary configuration was added to OSM. Then, the NS and VNF packages were created. The NS package describes a simple Kubernetes-based VNF (KNF) connected to a management network, as there is no need for a more complex slice. The VNF package, which in this case is a KNF, details which Helm chart should be used and sets up a day-1 action deploying another Helm chart. The first Helm chart is local and relates to the Falco deployment. It includes all configurations and rules. The second Helm chart refers to falco-exporter [4], which includes all required configurations. It also has a Prometheus job template, which is injected into Prometheus when the exporter is deployed. The exporter works as a bridge between the Falco instance and the event aggregator, which, in this case, is Prometheus. Thus, it is necessary to use Google Remote Procedure Call (gRPC) output, as it subscribes to the Falco instance.

Architecture Once more, Falco runs on the cluster, and the event aggregator, falco-exporter, runs on the machine hosting OSM. The Falco instance outputs its data to a gRPC server, which falco-exporter subscribes to.

In addition, falco-exporter is added as a Prometheus data source, meaning that Prometheus will scrape it. Considering OSM also boasts of a Grafana deployment, alerts are visible there. However, another service could communicate with the exporter or with Prometheus. This functions as proof of the system's applicability in a real-life setting, as any other data visualizer or data processor could be put in place.

A schematization of this organization is visible in Figure 2.

As is visible, data flows are also defined. When an action matching the rules goes through the kernel, Falco logs it. Considering that the gRPC output is enabled, it is also sent to the gRPC server. In parallel, the exporter, hosted on the MEO, is subscribed to the gRPC server, meaning that it gets populated with the data on the gRPC server. Additionally, Prometheus is set up to scrape the exporter in one-second intervals, meaning that, every second, the exporter's

[3] `https://falco.org/`

[4] `https://github.com/falcosecurity/falco-exporter`, last accessed July 1, 2026

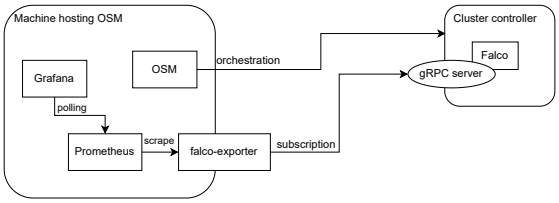

Fig. 2. Scenario 1 implementation

data is read by Prometheus. With the data on Prometheus, Grafana is able to graph it. No timings are specified regarding Grafana, as it polls its data sources.

The first step in the deployment of the system is to add the Kubernetes cluster to OSM, a process that involves configuring the cluster to be recognized and managed by OSM.

Once this is ensured, the VNF and NS packages are created. Then, an NS instance is created on the specified cluster.

This means that Falco is deployed on the cluster with the correct output settings, and falco-exporter is deployed on the machine hosting OSM. Moreover, settings are injected into the OSM Prometheus instance, enabling it to scrape the exporter. Thus, data is visible on Grafana. This adaptability of the system, which allows for the addition of other systems to gain a clearer view of the cluster's behavior, reassures us of its flexibility.

4.2 Scenario 2: Multi-Node Implementation

Now that the solution has been validated for its base use case, the second must be implemented — that of an orchestrated system with a more complicated cluster.

Implementation The current solution is extremely similar to the one described earlier. Its main differentiator is that the worker node differs from the cluster controller. Instead, it is a bare metal worker.

The main difference between the current architecture and the previous single-node implementation is that the cluster has an additional worker, which does not change the overall organization. However, it changes the overview of the system. In this organization, the gRPC external endpoint is clearly separated from the cluster. Previously, there was no need to differentiate it from the controller's endpoint, as there was no clear load-balancing from the external point to the cluster's internal gRPC servers. Thus, the external endpoint and the internal server were conflated into one entity. However, in this case, there are two. Therefore, the Kubernetes load-balancing is apparent. A complete overview of the system architecture is presented in Figure 3.

However, falco-exporter only subscribes once to the internal gRPC servers. Thus, if there is more than one gRPC server, as in this case, its data is not

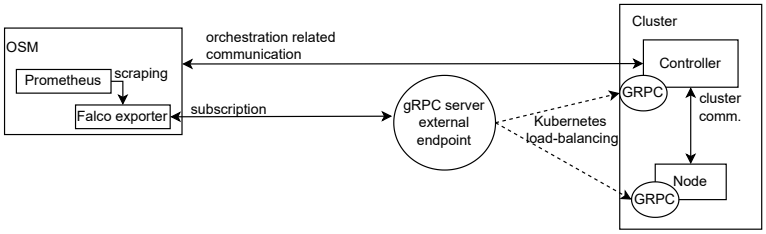

Fig. 3. Scenario 2 implementation

exported, as load-balancing only occurs once. The development team was alerted to this problem via a GitHub issue [5]. There has been no response, and one was created under the scope of this work.

5 Results and Discussion

The proposed system is evaluated in two main areas: system behavior and scalability. The first area focuses on assessing the system's performance, reliability, and accuracy in detecting alerts. Specifically, we investigate whether the system is resource-intensive, prone to errors, or experiences delays in alert detection. The second area evaluates the system's scalability when observing multiple nodes, comparing the performance of the single-node (SN) and multi-node (MN) configurations.

To perform these evaluations, we employ a two-step testing approach. First, we establish the baseline performance metrics for each machine involved in the test. Then, we remeasure these metrics while the system is running, allowing us to compare the results and quantify the overhead introduced by the solution. It is essential to note that the SN solution consists of a single machine, including its controller. In contrast, the MN solution comprises two machines: a VM controller and a bare metal worker machine. The specifications of each machine are listed in Table 1.

This testing methodology enables us to assess the system's behavior and scalability comprehensively and systematically, providing valuable insights into its performance and limitations.

We measured two key performance metrics to quantify the solution's footprint on each machine: CPU and memory usage. These metrics were collected using a custom script executed on each machine, leveraging the `top` command to retrieve CPU utilization and the `free` command to obtain memory usage. The script sampled these values at one-second intervals, providing a fine-grained view of the system's behavior.

The CPU usage values are tabulated in Table 2.

[5] `https://github.com/falcosecurity/falco-exporter/issues/97`, last accessed July 1, 2026

	SN Controller	MN Controller	MN Worker
RAM	8 GiB	8GiB	16 GB
# of processors	4 cores	4 cores	8 cores
Memory	64GiB	64 GiB	64 GB
Operating System	Ubuntu 20.04.4	Ubuntu 20.04.4	Debian 11

Table 1. Overview of each machine's specifications

	Average CPU usage	# of measurements
Idle SN	1.6%	1296
Running SN	3.1%	1363
Idle C MN	1.6%	1842
Running C MN	4.3%	2371
Idle W MN	0.45%	5268
Running W MN	0.84%	1906

Table 2. CPU usage results

An examination of the CPU usage results reveals some variance in the MN controller's performance, which can be attributed to the fact that VM is hosted on shared hardware, introducing inherent noise into the measurements. Nevertheless, it is clear that running the solution on all machines increases CPU usage.

This increase is expected, as no solution can operate without some level of overhead. The key consideration is whether this increase is significant enough to cause performance issues. In the case of the MN controller, which has the largest growth, the increase in CPU usage is moderate, rising by 2.7% overall. Although this increase is not negligible, it is unlikely to have a substantial impact on system performance. Considering that this is the most significant increase in CPU usage, it is clear that CPU usage should not be a problem for most machines running the system.

The memory usage results are tabulated in Table 3.

	Average memory usage	# of measurements
Idle SN	930MiB	1296
Running SN	999MiB	1363
Idle C MN	1615MiB	1842
Running C MN	1666MiB	2371
Idle W MN	1752MiB	5268
Running W MN	2075MiB	1906

Table 3. Memory usage results

A comparison of idle values for the SN and MN controllers reveals a notable difference, likely due to the inherent memory usage associated with adding nodes

to a cluster. However, this difference does not reflect the consumption of resources in the solution. Furthermore, the difference between the running values of the SN and MN controllers is similar, indicating that the solution's memory usage is consistent across different cluster sizes.

In contrast, the worker node exhibits a significant increase in memory usage when transitioning from an idle to a running state. A hypothesis was formed that this increase was due to the exporter establishing multiple connections. This hypothesis was confirmed through a test run in which no additional connections were made. The results of this test are not presented in this paper, as they were obtained from a system without alerts from all nodes, thus meaning that the system was not fully functional.

Taking into account both CPU and memory usage, two key conclusions can be drawn. Firstly, the alerting solution has a minimal impact on the observed system, demonstrating its lightweight nature. Secondly, while there is some difference in resource consumption between smaller and larger clusters, this difference is relatively small, indicating that the solution scales well with increasing cluster size. Still, future work should include further testing with bigger clusters, as the current approach is necessarily limited.

While resource consumption is an important consideration, it is not the only metric that can impact the overall utility of the system. Deployment time, for instance, can significantly affect the system's usefulness. If the deployment process is too long, the benefits of the system may be outweighed by the delay. Furthermore, the system's scalability is also a crucial factor, particularly when dealing with larger clusters.

To evaluate the deployment time of our solution, we conducted a series of experiments. The SN system was deployed 816 times, with an average deployment time of 36.95 seconds. In contrast, the MN system was deployed 376 times, with an average deployment time of 47 seconds. These results suggest that adding a node to the cluster increases the deployment time, although this may also be attributed to the slower communication between nodes due to the bare metal machine being on a different network.

The deployment times are graphed in Figure 4, revealing a similar pattern for both systems. This suggests that the primary difference between the two systems is a fixed added value, rather than a proportional increase in deployment time. This finding has important implications for the system's scalability and overall utility. In theory, this means that every additional node in the cluster will add a set number of seconds to the deployment time, meaning that there will be a point at which so many nodes will be in the cluster that the system will take too long to deploy.

The deployment of our system demonstrated exceptional reliability, with zero failures across all deployments. Notably, the system successfully generated thousands of alerts without any instances of lost or false alerts, thereby achieving a perfect detection rate with no false positives or false negatives.

The average time of action and the alert population remained consistent, regardless of the volume of alerts, as shown in Figure 5. The data suggests that

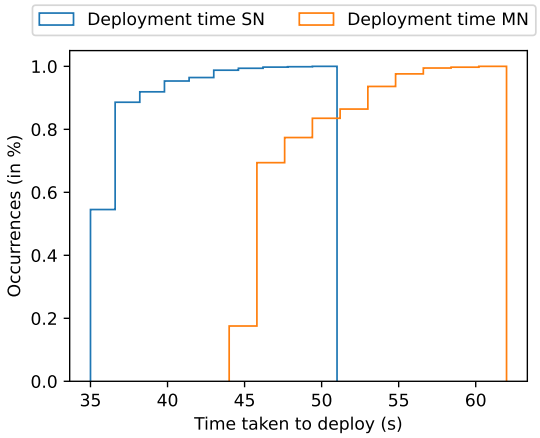

Fig. 4. Deployment times

smaller volumes of alerts may result in longer average times; however, that is a mistake. Considering that alerts are scraped from the gRPC server, this average time has the scrape interval embedded. If there is a small volume of alerts, the scrape interval is more noticeable, as it is not an overhead spread across multiple alerts.

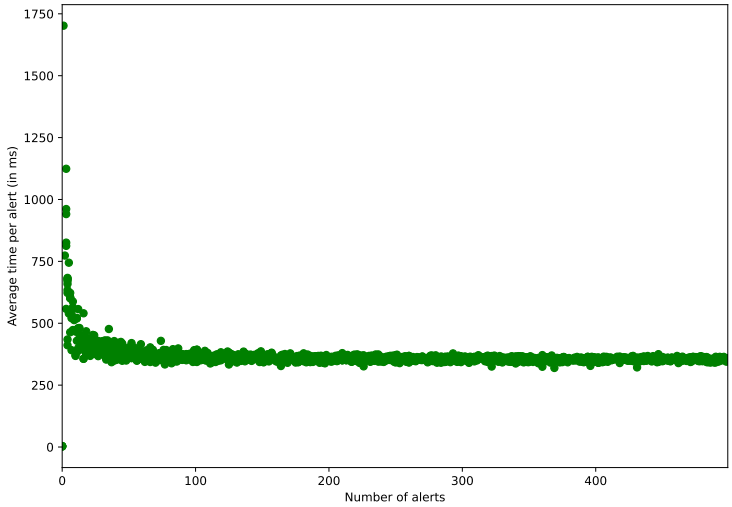

Fig. 5. Average time of action and alert, by volume of alerts

The system's effectiveness is tied to the quality of the defined rules. Falco provides a robust set of base rules, which we supplemented with additional rules to test the system's capabilities and ease of rule creation. The rules are declarative, with a syntax similar to Python, and are defined as a `YAML` object with six fields: rule name, description, condition, output, priority, and tags. The condition field defines the rule behavior, and Falco's access to kernel-level information enables the detection of a wide range of behaviors. An example of an alert is presented in Figure 6, which demonstrates the comprehensive data available for incident analysis.


```
{"hostname":"falco-2dwcj","output":"10:31:48.409094474: Warning Sensitive file opened for reading by non-trust
ed program (file=/etc/shadow gparent=bash ggparent=node gggparent=node evt_type=openat user=root user_uid=0 us
er_loginuid=1000 process=cat proc_exepath=/usr/bin/cat parent=sudo command=cat /etc/shadow terminal=34817 cont
ainer_id=host container_image=<NA> container_image_tag=<NA> container_name=host k8s_ns=<NA> k8s_pod_name=<NA>)
","priority":"Warning","rule":"Read sensitive file untrusted","source":"syscall","tags":["T1555","container","
filesystem","host","maturity_stable","mitre_credential_access"],"time":"2024-02-20T10:31:48.409094474Z", "outp
ut_fields": {"container.id":"host","container.image.repository":null,"container.image.tag":null,"container.nam
e":"host","evt.time":1708425108409094474,"evt.type":"openat","fd.name":"/etc/shadow","k8s.ns.name":null,"k8s.p
od.name":null,"proc.aname[2]":"bash","proc.aname[3]":"node","proc.aname[4]":"node","proc.cmdline":"cat /etc/sh
adow","proc.exepath":"/usr/bin/cat","proc.name":"cat","proc.pname":"sudo","proc.tty":34817,"user.loginuid":100
0,"user.name":"root","user.uid":0}}
```


Fig. 6. Alert when reading `etc/shadow`

Furthermore, all alerts are forwarded to a Prometheus timeseries, giving administrators a centralized view of system activity. This enables them to monitor their cluster from the orchestrator and take necessary measures to enhance system security.

6 Conclusion

In this work, we present a host-centric approach to securing workflow execution in edge environments by observing the runtime behavior of MEC hosts. As workflows become increasingly distributed and dependent on infrastructure beyond the traditional data center, ensuring the integrity and trustworthiness of edge hosts becomes critical. Our system addresses this challenge by continuously monitoring host activity and triggering alerts based on violations of predefined behavioral rules. This lightweight mechanism avoids the complexity of full attestation or hardware-based trust anchors while still providing timely and actionable insight into potential compromises or misconfigurations. Through our prototype and evaluation, we demonstrate that observation is both feasible and effective for detecting anomalous conditions in edge systems with minimal performance impact. Moving forward, we see opportunities to expand this work with automatic rule creation, and host removal after clear signs of compromise. By shifting attention to the behavior of the execution environment itself, our approach offers a practical and scalable path to securing workflows at the edge.

Acknowledgments. This work has been partly developed in the scope of the project EXIGENCE. EXIGENCE has received funding from the Smart Networks and Services Joint Undertaking (SNS JU) under the European Union's Horizon Europe research

and innovation programme under Grant Agreement No 101139120. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or Smart Networks and Services Joint Undertaking. Neither the European Union nor the granting authority can be held responsible for them.

References

[1] Pasika Ranaweera, Anca Delia Jurcut, and Madhusanka Liyanage. "Survey on Multi-Access Edge Computing Security and Privacy". In: *IEEE Communications Surveys and Tutorials* 23.2 (2021), pp. 1078–1124. ISSN: 1553877X. DOI: 10.1109/COMST.2021.3062546. URL: https://ieeexplore.ieee.org/document/9364272/.

[2] Meisong Wang et al. "An overview of cloud based content delivery networks: Research dimensions and state-of-the-Art". In: *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 9070 (2015), pp. 131–158. ISSN: 16113349. DOI: 10.1007/978-3-662-46703-9_6.

[3] Dario Sabella, Alex Reznik, and Rui Frazao. *Multi-Access Edge Computing in Action*. CRC Press, 2019. ISBN: 9780429056499. DOI: 10.1201/9780429056499. URL: https://www.taylorfrancis.com/books/9780429508752.

[4] Michael Armbrust et al. *Above the clouds: A berkeley view of cloud computing*. Tech. rep. Technical Report UCB/EECS-2009-28, EECS Department, University of California, 2009.

[5] Keyan Cao et al. "An Overview on Edge Computing Research". In: *IEEE Access* 8 (2020), pp. 85714–85728. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2020.2991734. URL: https://ieeexplore.ieee.org/document/9083958/.

[6] Daniele Bringhenti et al. "Automation for Network Security Configuration: State of the Art and Research Trends". In: *ACM Computing Surveys* 56.3 (2024), pp. 1–37. ISSN: 0360-0300. DOI: 10.1145/3616401. URL: https://dl.acm.org/doi/10.1145/3616401.

[7] Alejandro Fornés-Leal et al. "Evolution of MANO Towards the Cloud-Native Paradigm for the Edge Computing". In: *Advanced Computing and Intelligent Technologies*. Ed. by Rabindra Nath Shaw et al. Singapore: Springer Nature Singapore, 2022, pp. 1–16. ISBN: 978-981-19-2980-9.

[8] Sebastian Burckhardt et al. "Netherite: efficient execution of serverless workflows". In: *Proceedings of the VLDB Endowment* 15.8 (2022), pp. 1591–1604. ISSN: 2150-8097. DOI: 10.14778/3529337.3529344. URL: https://dl.acm.org/doi/10.14778/3529337.3529344.

[9] William Hurst and Áine MacDermott. "Evaluating the effects of cascading failures in a network of critical infrastructures". In: *International Journal of System of Systems Engineering* 6.3 (2015), p. 221. ISSN: 1748-0671. DOI: 10.1504/IJSSE.2015.071458. URL: http://www.inderscience.com/link.php?id=71458.

[10] Maha Aljohani. "Transforming data-intensive workflows: A cutting-edge multi-layer security and quality aware security framework". In: *Concurrency and Computation: Practice and Experience* 36.13 (2024). DOI: 10.1002/cpe.8068.

[11] Keyan Cao et al. "An Overview on Edge Computing Research". In: *IEEE Access* 8 (2020), pp. 85714–85728. ISSN: 21693536. DOI: 10.1109/ACCESS.2020.2991734.

[12] Huang Zeyu et al. "Survey on Edge Computing Security". In: *2020 International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*. IEEE, 2020, pp. 96–105. ISBN: 978-1-7281-6499-1. DOI: 10.1109/ICBAIE49996.2020.00027. URL: https://ieeexplore.ieee.org/document/9196442/.

[13] Muhammad Usman et al. "A Survey on Observability of Distributed Edge & Container-Based Microservices". In: *IEEE Access* 10 (2022), pp. 86904–86919. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2022.3193102. URL: https://ieeexplore.ieee.org/document/9837035/.

[14] Salman Taherizadeh et al. "Monitoring self-adaptive applications within edge computing frameworks: A state-of-the-art review". In: *Journal of Systems and Software* 136 (2018), pp. 19–38. ISSN: 01641212. DOI: 10.1016/j.jss.2017.10.033.

[15] Jiale Zhang et al. "Data Security and Privacy-Preserving in Edge Computing Paradigm: Survey and Open Issues". In: *IEEE Access* 6 (2018), pp. 18209–18237. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2018.2820162. URL: http://ieeexplore.ieee.org/document/8327600/.

[16] Vivek Kumar Prasad, Madhuri D Bhavsar, and Sudeep Tanwar. "Influence of Montoring: Fog and Edge Computing". In: *Scalable Computing: Practice and Experience* 20.2 (2019), pp. 365–376. ISSN: 1895-1767. DOI: 10.12694/scpe.v20i2.1533. URL: https://www.scpe.org/index.php/scpe/article/view/1533.

[17] Álvaro Brandón et al. "FMonE: A Flexible Monitoring Solution at the Edge". In: *Wireless Communications and Mobile Computing* 2018 (2018), pp. 1–15. ISSN: 15308677. DOI: 10.1155/2018/2068278.

[18] Marco Gaglianese et al. "Lightweight Self-adaptive Cloud-IoT Monitoring across Fed4FIRE+ Testbeds". In: *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 2022, pp. 1–6. ISBN: 978-1-6654-0926-1. DOI: 10.1109/INFOCOMWKSHPS54753.2022.9798259. URL: https://ieeexplore.ieee.org/document/9798259/.

[19] Vera Colombo et al. "Towards self-adaptive peer-to-peer monitoring for fog environments". In: *Proceedings of the 17th Symposium on Software Engineering for Adaptive and Self-Managing Systems*. New York, NY, USA: ACM, 2022, pp. 156–166. ISBN: 9781450393058. DOI: 10.1145/3524844.3528055. URL: https://dl.acm.org/doi/10.1145/3524844.3528055.

[20] Shashikant Ilager et al. "DEMon: Decentralized Monitoring for Highly Volatile Edge Environments". In: *Proceedings - 2022 IEEE/ACM 15th International Conference on Utility and Cloud Computing, UCC 2022* null

(2022), pp. 145–150. DOI: 10.1109/UCC56403.2022.00026. URL: https://ieeexplore.ieee.org/document/10061772/.

[21] Saad Khoudali, Karim Benzidane, and Abderrahim Sekkaki. "EMMCS: An Edge Monitoring Framework for Multi-Cloud Environments using SNMP". In: *International Journal of Advanced Computer Science and Applications* 10.1 (2019). ISSN: 21565570. DOI: 10.14569/IJACSA.2019.0100178. URL: http://thesai.org/Publications/ViewPaper?Volume=10\&Issue=1\&Code=ijacsa\&SerialNo=78.

[22] Muhammad Usman and JongWon Kim. "SmartX Multi-View Visibility Framework for unified monitoring of SDN-enabled multisite clouds". In: *Transactions on Emerging Telecommunications Technologies* 33.8 (2022). ISSN: 2161-3915. DOI: 10.1002/ett.3819. URL: https://onlinelibrary.wiley.com/doi/10.1002/ett.3819.

[23] Arthur Souza et al. "Osmotic Monitoring of Microservices between the Edge and Cloud". In: *2018 IEEE 20th International Conference on High Performance Computing and Communications; IEEE 16th International Conference on Smart City; IEEE 4th International Conference on Data Science and Systems (HPCC/SmartCity/DSS)*. IEEE, 2018, pp. 758–765. ISBN: 978-1-5386-6614-2. DOI: 10.1109/HPCC/SmartCity/DSS.2018.00129. URL: https://ieeexplore.ieee.org/document/8622867/.

[24] Andres Freund. *oss security — backdoor in upstream xz/liblzma leading to ssh server compromise*. 2024. URL: https://www.openwall.com/lists/oss-security/2024/03/29/4 (visited on 04/08/2024).